

Hourly Electricity Demand Forecasting

Training, Fine-tuning, and Comparative Evaluation
of Time-series Foundation Models

Dataset	Monash electricity_hourly
Scope	321 series, 26,304 hours per series
Forecasting task	5 windows $\times$ 168 hours
Primary metric	MASE ($m = 24$)
Recommended model	chronos_bolt_base_ft
Compute environment	NVIDIA A100-SXM4-40GB

Executive Summary

This report documents the training design and experimental results of an end-to-end benchmark on hourly electricity consumption. The Monash `electricity_hourly` dataset contains 321 equal-length series covering 2012–2014, with 26,304 hourly observations per series. Every method forecasts the next week (168 hours) in five non-overlapping rolling-origin windows, producing 269,640 series-hour test predictions.

The comparison includes seasonal-naïve references, AutoTheta, AutoETS, MSTL, a global LightGBM, PatchTST trained from scratch, and Chronos-Bolt in both zero-shot and fine-tuned forms. All methods share the same data splits, post-processing, and centralized scoring code. The primary point metric is MASE with daily seasonality; probabilistic performance is evaluated with Weighted Quantile Loss over nine quantiles.

Main result

`chronos_bolt_base_ft` ranks first with **0.9358 MASE** and **0.0896 WQL**. It reduces MASE by 15.9 % relative to the strong “repeat last week” baseline. All three Chronos-Bolt variants beat that baseline, while the classical models, global LightGBM, and PatchTST trained from scratch do not. Fine-tuning provides only a small average gain, but lowers MASE from 1.3165 to 1.2821 in the hardest Christmas-week window.

Calibration remains an important limitation. The nominal 80 % Chronos prediction intervals achieve only about 70 %–71 % empirical coverage. A past-only conformal correction is therefore recommended when coverage guarantees matter.

Contents

Executive Summary	i
1 Objective and Scope	1
2 Dataset and Forecasting Protocol	1
2.1 Dataset structure	1
2.2 Rolling-origin evaluation	1
2.3 Leakage prevention and common scoring	2
3 Models Compared	2
3.1 Seasonal-naive and classical models	2
3.2 Global LightGBM	2
3.3 PatchTST trained from scratch	3
3.4 Chronos-Bolt zero-shot and fine-tuned	3
4 Training and Fine-tuning	3
4.1 PatchTST training	3
4.2 Chronos-Bolt fine-tuning	3
5 Evaluation Metrics	4
5.1 Point metrics	4
5.2 Probabilistic metrics	5
6 Experimental Results	5
6.1 Overall ranking	5
6.2 Window-level results and the Christmas effect	5
6.3 What fine-tuning contributes	7
6.4 Breadth of gains and scale sensitivity	7
6.5 Probabilistic calibration	7
7 Compute Environment and Reproducibility	8
8 Limitations and Validity Threats	8
9 Conclusion and Recommended Use	9
A Detailed Hyperparameters	9
B Artifact Traceability	10

1 Objective and Scope

The benchmark asks whether a pretrained time-series foundation model provides a practical advantage for hourly electricity-demand forecasting under a strong and leakage-free evaluation protocol. Four research questions guide the analysis:

1. Does Chronos-Bolt beat the weekly seasonal-naive baseline?
2. Does pretraining outperform a Transformer trained from scratch on the same data?
3. Does light dataset-specific fine-tuning improve zero-shot Chronos-Bolt?
4. Do gains in point accuracy extend to probabilistic accuracy and interval calibration?

The numerical sources are the frozen result tables in `outputs/metrics/`. The implementation source is `energy_forecasting_foundation_models.ipynb`. This report does not run a new benchmark; it explains the completed run using its model configuration, training logs, forecast-derived metrics, and recorded environment metadata.

2 Dataset and Forecasting Protocol

2.1 Dataset structure

The `electricity_hourly` dataset is distributed through the Monash Time Series Forecasting Archive [2]. It is an hourly, aggregated version of the UCI *ElectricityLoadDiagrams20112014* collection. The raw `.tsf` file is converted to the long schema `unique_id`, `ds`, `y`, then pivoted into a shared timestamp-by-series matrix.

Table 1: Dataset characteristics.

Property	Value
Number of series	321
Timestamps per series	26,304
Total observations	8,443,584
Date range	January 1, 2012–December 31, 2014
Frequency	Hourly
Missing values	None
Series lengths	Equal
Unit	kW, as defined by the source dataset

Series magnitudes span several orders of magnitude, and both daily ($m = 24$) and weekly ($m = 168$) seasonality are strong. These properties motivate per-series normalization for global models, daily-scaled MASE for evaluation, and the weekly seasonal naive as the operational bar.

2.2 Rolling-origin evaluation

Each model forecasts 168 hours for all 321 series. Five non-overlapping test windows are created by walking backward from the end of the shared index. For every window, training and validation data end at or before that window’s cutoff.

Table 2: Frozen rolling-origin windows.

Window	Cutoff	Forecast period
1	November 26, 2014 23:00	November 27–December 3
2	December 3, 2014 23:00	December 4–10
3	December 10, 2014 23:00	December 11–17
4	December 17, 2014 23:00	December 18–24
5	December 24, 2014 23:00	December 25–31

2.3 Leakage prevention and common scoring

Model code cannot access the metrics. Each method implements a common interface that receives only past observations and returns nine quantile forecasts for 168 hours. Normalization statistics, validation tails, clipping thresholds, and training samples are all computed from past-only data.

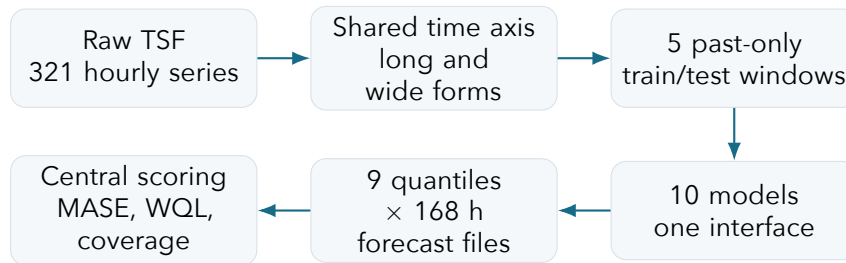


Figure 1: Common evaluation pipeline and leakage controls.

Demand forecasts are clipped to be non-negative. A per-series upper bound of ten times the maximum past value limits numerical explosions. The nine quantiles are sorted at every timestamp to prevent quantile crossing. These post-processing steps are identical for all models.

3 Models Compared

3.1 Seasonal-naive and classical models

Two seasonal-naive methods repeat the last day ($m = 24$) or last week ($m = 168$). The weekly variant is the main operational baseline. Because these point forecasts are copied into all nine quantile columns, their prediction intervals are degenerate and should not be interpreted as genuine uncertainty estimates.

AutoTheta, AutoETS, and MSTL are fit independently to each series using the most recent 2,016 hours. MSTL decomposes daily and weekly seasonal components and uses ETS as the trend forecaster. Gaussian prediction intervals are mapped to the nine evaluation quantiles.

3.2 Global LightGBM

LightGBM [4] is trained globally across all series. The target is transformed with a local per-series standard scaler. Features include lags of 1, 24, 48, 168, and 336 hours; daily and weekly rolling statistics; hour; day of week; month; and a weekend indicator. Separate models are fit for $q_{0.1}$, $q_{0.5}$, and $q_{0.9}$, with intermediate quantiles obtained by linear interpolation.

3.3 PatchTST trained from scratch

PatchTST segments the time axis into overlapping patches and shares Transformer weights across channels [3]. The implementation uses 512 hours of context, patch length 16, stride 8, three encoder layers, hidden size 128, eight attention heads, and RevIN normalization. Nine quantiles are learned jointly with a multi-quantile pinball loss.

3.4 Chronos-Bolt zero-shot and fine-tuned

Chronos is a family of probabilistic foundation models pretrained on broad collections of time series [1]. The saved `chronos-bolt-base` configuration accepts up to 2,048 historical steps, divides them into non-overlapping patches of length 16, applies instance normalization, and passes the result through a T5-based encoder-decoder. The configuration contains 12 encoder and 12 decoder layers, model width 768, 12 attention heads, feed-forward width 3,072, and dropout 0.1. Its output head directly predicts nine quantiles for the next 64 steps.

The 168-hour horizon is generated in 64-hour blocks. Where the library does not natively allow a longer prediction length, each block’s median forecast is appended to the context before producing the next block. This median-feedback mechanism is an important potential source of accumulated error at the end of the horizon.

Table 3: Main configurations of the learned models.

Property	LightGBM	PatchTST	Chronos-Bolt Base
Context	8,760 hours	512 hours	2,048 hours
Forecast output	3 quantiles + interpolation	168 h, 9 quantiles	64 h, 9 quantiles
Normalization	Local standardization	RevIN	Instance normalization
Core capacity	300 trees, 64 leaves	3 layers, $d = 128$	12+12 layers, $d = 768$
Loss	Quantile objective	Multi-quantile loss	Scaled multi-quantile loss
Calendar features	Yes	No	No
Pretraining	No	No	Yes

4 Training and Fine-tuning

4.1 PatchTST training

PatchTST is refit from scratch for every backtest window. The last 168 past hours form a validation tail. Training uses a learning rate of 10^{-3} , batch size 32 series, and 256 sampled windows per step. Validation is checked every 100 steps; training stops after five consecutive checks without improvement. The 50,000-step limit is only a safety ceiling and is never approached.

4.2 Chronos-Bolt fine-tuning

For each backtest window, the model is reinitialized from `amazon/chronos-bolt-base`. A training example consists of a randomly chosen series and time endpoint, a 2,048-hour context, and the following 64-hour target. The final 64 past hours are excluded from sampling and held out for validation. All model weights are updated with AdamW at learning rate 10^{-5} and batch size 24; the gradient norm is clipped at 1.0.

Validation is evaluated every 50 steps. If loss fails to improve by more than 10^{-5} for five consecutive checks, training stops and the best checkpoint is restored. Each backtest window is fine-tuned independently from the pretrained weights. The shareable final model is then trained on the full history with the same recipe.

Table 4: Observed early-stopping steps.

Model	W1	W2	W3	W4	W5	Full data
PatchTST	1,800	2,500	1,600	2,300	2,000	–
Chronos-Bolt fine-tuning	500	350	650	500	500	400

For the full-data fine-tune, the minimum validation loss occurs at step 150. The loop terminates at step 400 after exhausting patience, then restores the weights from step 150. Therefore, “400 steps” describes the duration of the early-stopping loop, not the final checkpoint’s weight state.

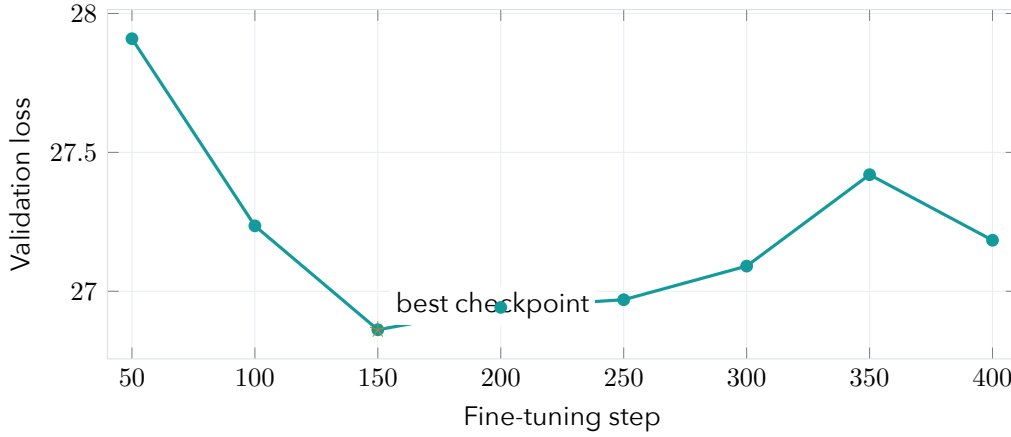


Figure 2: Validation curve for the full-data Chronos-Bolt fine-tune.

5 Evaluation Metrics

Let $y_{t,i}$ denote the observed value, $\hat{y}_{t,i}$ the median forecast, T_i the training sample for series i , and $m = 24$ the seasonal period. Metrics are computed first for each series-window pair and then averaged equally over 321 series and five windows. This prevents high-consumption series from dominating the result.

5.1 Point metrics

$$\text{MAE}_i = \frac{1}{H} \sum_{t=1}^H |y_{t,i} - \hat{y}_{t,i}|, \quad (1)$$

$$\text{RMSE}_i = \sqrt{\frac{1}{H} \sum_{t=1}^H (y_{t,i} - \hat{y}_{t,i})^2}, \quad (2)$$

$$\text{sMAPE}_i = \frac{100}{H} \sum_{t=1}^H \frac{2|y_{t,i} - \hat{y}_{t,i}|}{|y_{t,i}| + |\hat{y}_{t,i}|}, \quad (3)$$

$$\text{MASE}_i = \frac{\text{MAE}_i}{\frac{1}{|T_i| - m} \sum_{t=m+1}^{|T_i|} |y_{t,i} - y_{t-m,i}|}. \quad (4)$$

MASE is scale-free and supports comparisons across series with different magnitudes [5]. Seasonal-constant series with a zero denominator are excluded from MASE and counted explicitly. No series-window pair is excluded in this run.

5.2 Probabilistic metrics

For quantile $q \in \{0.1, \dots, 0.9\}$, pinball loss is

$$L_q(y, \hat{y}_q) = \max\{q(y - \hat{y}_q), (q - 1)(y - \hat{y}_q)\}. \quad (5)$$

The project computes Weighted Quantile Loss as

$$\text{WQL}_i = \frac{2}{9 \sum_t |y_{t,i}|} \sum_q \sum_{t=1}^H L_q(y_{t,i}, \hat{y}_{q,t,i}). \quad (6)$$

Lower WQL is better. Central 80 % interval coverage is the fraction of observations for which $q_{0.1} \leq y \leq q_{0.9}$. The nominal target is 0.80. High coverage alone is not sufficient: unnecessarily wide intervals can cover well while scoring poorly on WQL.

6 Experimental Results

6.1 Overall ranking

Table 5 reports all ten models under the common protocol. Models are ranked by the primary metric, MASE.

Table 5: Overall leaderboard. Down arrows indicate that lower values are better.

Model	MASE↓	sMAPE↓	RMSE↓	MAE↓	WQL↓	Coverage	vs. baseline
chronos_bolt_base_ft	0.9358	10.83	223.46	150.15	0.0896	0.701	−15.9%
chronos_bolt_base	0.9378	10.68	224.03	149.06	0.0900	0.712	−15.7%
chronos_bolt_small	0.9754	11.11	237.02	159.25	0.0920	0.710	−12.4%
seasonal_naive_168	1.1130	11.71	276.12	174.21	0.1279	0.095	baseline
lgbm_global	1.2073	14.40	280.71	191.16	0.2828	0.974	+8.5%
mstl	1.2984	17.01	299.27	229.53	0.1398	0.884	+16.7%
seasonal_naive_24	1.2993	14.33	315.09	207.06	0.1527	0.045	+16.7%
auto_ets	1.3988	17.40	310.21	223.10	0.2335	0.808	+25.7%
patchtst	1.7116	20.10	378.90	300.10	0.1517	0.721	+53.8%
auto_theta	1.7528	19.93	464.24	362.77	0.2337	0.956	+57.5%

Three findings stand out:

- Only the Chronos-Bolt family beats the weekly seasonal-naïve baseline on MASE.
- Fine-tuned and zero-shot Chronos-Bolt Base improve MASE by 15.9 % and 15.7 %, respectively, relative to that baseline.
- The WQL gap is larger: 0.0896 for fine-tuned Chronos is 35.9 % lower than MSTL’s 0.1398, the best WQL among the reasonably calibrated classical alternatives.

Fine-tuned Chronos ranks first on MASE, RMSE, and WQL, but zero-shot Chronos Base is slightly better on sMAPE and MAE. The results therefore do not support a claim of superiority on every metric.

6.2 Window-level results and the Christmas effect

Every model deteriorates in the December 25–31 window. PatchTST’s MASE rises to 3.474, while Chronos remains between 1.28 and 1.32. PatchTST sees only past target values and has no explicit

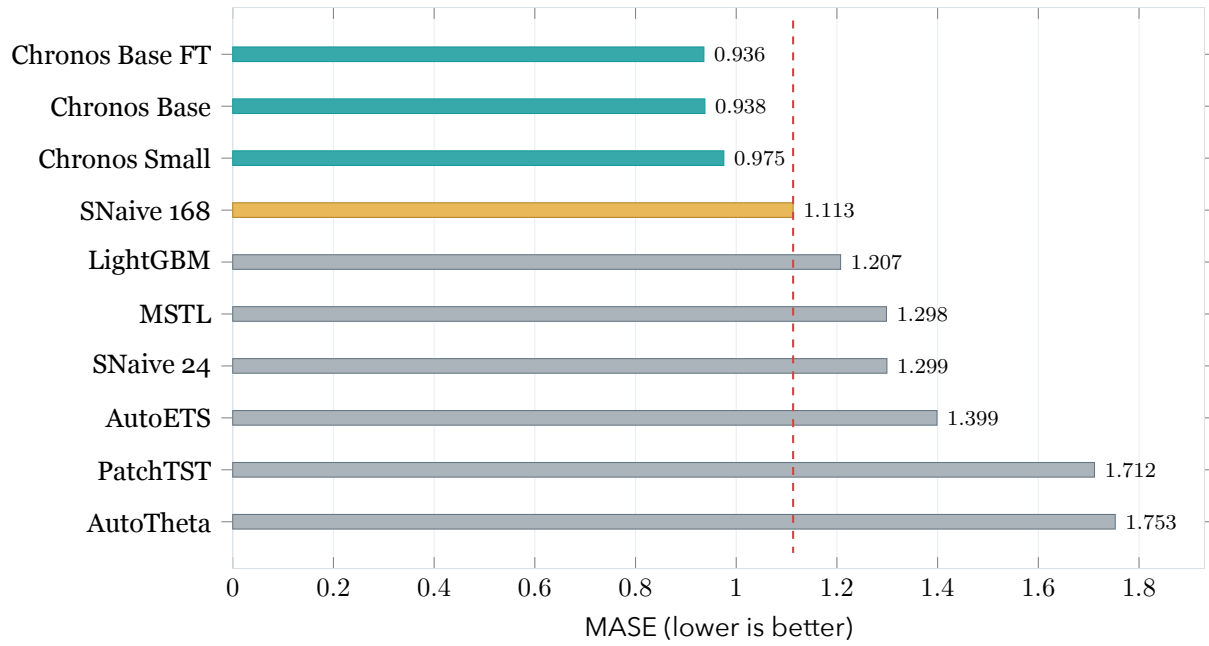


Figure 3: Overall MASE. The dashed line is the weekly seasonal-naive baseline.

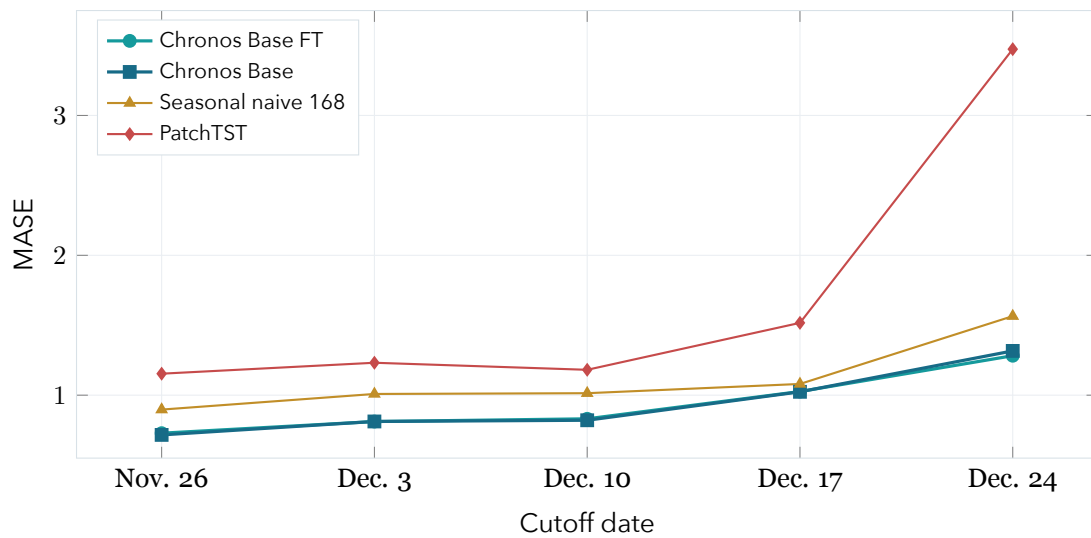


Figure 4: MASE across the five backtest windows. The final cutoff forecasts December 25–31.

Table 6: Window-level MASE for selected models.

Model	Nov. 26	Dec. 3	Dec. 10	Dec. 17	Dec. 24
Chronos Base FT	0.728	0.812	0.831	1.026	1.282
Chronos Base	0.716	0.812	0.821	1.024	1.317
Seasonal naive 168	0.897	1.009	1.014	1.080	1.565
LightGBM	0.910	1.051	1.120	1.197	1.759
PatchTST	1.154	1.232	1.181	1.517	3.474

calendar input, making the holiday regime change difficult to anticipate. Chronos also has no calendar covariate, so the gap cannot be attributed solely to calendar awareness. More general patterns learned in pretraining and earlier holiday episodes in the fine-tuning data are plausible explanations.

6.3 What fine-tuning contributes

The overall difference between fine-tuned and zero-shot Chronos-Bolt Base is only 0.0020 MASE, about 0.22 %, and 0.0004 WQL. Across the first four windows, the fine-tuned minus zero-shot MASE differences are +0.0123, −0.0005, +0.0104, and +0.0021. Fine-tuning is therefore roughly level or slightly worse on ordinary weeks. In the final window, however, the difference is −0.0344, a relative MASE improvement of about 2.6 %.

At the series level in the Christmas window, fine-tuning produces lower MASE for 187 of 321 series (58.3 %). Under equal win probability, the exact one-sided sign-test result is $p = 0.0018$. The WQL win count is 183/321. Across all windows and series, however, fine-tuning beats zero-shot Base only about 44 % of the time.

Appropriate interpretation

The evidence does not support the claim that fine-tuning is better everywhere. A narrower claim is supported: *fine-tuning preserved average performance and was more robust in the single clearly anomalous test week*. This robustness observation should be validated on other years and exceptional periods.

6.4 Breadth of gains and scale sensitivity

Fine-tuned Chronos beats the weekly seasonal naive on 1,255 of 1,605 series-window pairs (78.2 %); zero-shot Base does so on 1,273 pairs (79.3 %). Fine-tuned Chronos beats PatchTST on 1,524 pairs (95.0 %). The average gain is therefore not driven by only a few high-volume series.

When series are grouped into terciles by mean consumption, fine-tuned Chronos achieves MASE values of 0.924, 0.949, and 0.934 for small-, medium-, and large-scale series. The corresponding weekly seasonal-naive values are 1.107, 1.138, and 1.094. Per-series normalization appears to support balanced performance across scales.

6.5 Probabilistic calibration

Table 7: Empirical coverage of nominal 80 % intervals.

Model group	Coverage	Interpretation
Chronos-Bolt family	0.70–0.71	Intervals are roughly 9–10 percentage points too narrow
PatchTST	0.721	Too narrow; coverage falls to 0.543 in the Christmas window
AutoETS	0.808	Near nominal, but point accuracy is weak
MSTL	0.884	Somewhat wide
LightGBM / AutoTheta	0.974 / 0.956	Far too wide; high coverage alone is not success
Seasonal naives	0.045–0.095	Degenerate because they are point forecasts

Chronos combines low WQL with undercoverage. Fine-tuning reduces coverage from 0.712 to 0.701. If production use requires reliable coverage, the $q_{0.1}-q_{0.9}$ interval should be widened with a series- or group-level conformal correction learned only from past calibration residuals.

7 Compute Environment and Reproducibility

The benchmark was run on an NVIDIA A100-SXM4-40GB with random seed 42. Recorded software versions are listed in table 8.

Table 8: Recorded run environment.

Component	Version / value
GPU	NVIDIA A100-SXM4-40GB
PyTorch	2.11.0+cu128
NumPy / pandas	2.0.2 / 2.2.2
StatsForecast / MLForecast	2.0.3 / 1.0.31
LightGBM	4.6.0
NeuralForecast	3.1.9
chronos-forecasting	1.5.3

The benchmark notebook performs data loading, window construction, training or inference for all ten models, centralized scoring, training-log export, and final model serialization. Documented end-to-end runtime on an A100 is about 1.5–2.5 hours. The final model is saved in Hugging Face format as `config.json` and `model.safetensors`; a reload-and-predict test verifies a 168-hour output for all 321 series.

To reproduce the run:

1. Open `energy_forecasting_foundation_models.ipynb` in a GPU environment.
2. Review the data path and frozen protocol before running models. Do not revise the protocol after inspecting results.
3. Run all cells in order. Archive `outputs/run_meta.json` and the leaderboard in `outputs/metrics/`.
4. Reload the saved model and evaluate coverage and distribution shift on independent data before production use.

Published model: [mskayacioglu/chronos-bolt-base-monash-electricity-hourly](https://huggingface.co/mskayacioglu/chronos-bolt-base-monash-electricity-hourly).

8 Limitations and Validity Threats

- **One dataset and one frequency:** Results are specific to 321 hourly electricity series and do not automatically generalize to other countries, climates, frequencies, or demand definitions.
- **Five test weeks:** The test period contains only the final five weeks of 2014. Other seasons, heat waves, price shocks, and holidays are not represented.
- **One anomalous episode:** The fine-tuning robustness gain is observed in one Christmas week and is not a multi-event study.
- **No exogenous variables:** Weather, holidays, prices, and customer information are not supplied to Chronos or PatchTST, despite their likely importance during exceptional periods.

- **Model-selection budget:** The protocol is frozen, but each model family does not receive an equally extensive hyperparameter search. Results compare reasonable recipes rather than each family’s theoretical optimum.
- **Interval calibration:** Chronos does not attain nominal 80 % coverage; uncalibrated use can understate uncertainty.
- **Long-horizon rolling:** Chronos extends its native 64-step output with median feedback, which can accumulate error in later blocks.

9 Conclusion and Recommended Use

The recommended model for this benchmark is `chronos_bolt_base_ft`. It ranks first on the primary MASE metric and WQL, performs consistently across scale groups, and is more robust than the zero-shot model in the hardest test week. However, `chronos_bolt_base` is only 0.002 MASE behind and requires no dataset-specific training, making it a strong lower-cost operational option. Where foundation-model deployment is unavailable, the weekly seasonal naive remains a reliable and surprisingly competitive fallback.

Decision summary

Highest benchmark accuracy	Fine-tuned Chronos-Bolt Base
Strong training-free option	Zero-shot Chronos-Bolt Base
Lowest-complexity fallback	Weekly seasonal naive
Required pre-production step	Backtest on new periods and calibrate quantiles
Priority extension	Add weather and holiday covariates; test multiple anomalous periods

A Detailed Hyperparameters

Table 9: Main implementation hyperparameters.

Model / component	Parameter	Value
General	Horizon / windows	168 hours / 5 windows
General	Quantiles	0.1, 0.2, ..., 0.9
General	Clipping	[0, 10 × past maximum]
Classical	Context	2,016 hours
MSTL	Seasonalities	24 and 168; ETS trend forecaster
LightGBM	Context	8,760 hours
LightGBM	Lags	1, 24, 48, 168, 336
LightGBM	Trees / leaves	300 / 64
LightGBM	Learning rate	0.05
LightGBM	Minimum child samples	200
LightGBM	Row / column sampling	0.8 / 0.9
PatchTST	Input / horizon	512 / 168
PatchTST	Patch / stride	16 / 8
PatchTST	Hidden / heads / layers	128 / 8 / 3
PatchTST	Dropout	0.2

Model / component	Parameter	Value
PatchTST	Learning rate	10^{-3}
PatchTST	Batch / window batch	32 / 256
PatchTST	Validation / patience	Every 100 steps / 5 checks
Chronos-Bolt	Context / native horizon	2,048 / 64
Chronos-Bolt	Patch / stride	16 / 16
Chronos-Bolt	Model width / heads	768 / 12
Chronos-Bolt	Encoder / decoder	12 / 12 layers
Chronos-Bolt	Feed-forward / dropout	3,072 / 0.1
Fine-tuning	Optimizer	AdamW, learning rate 10^{-5}
Fine-tuning	Batch / target	24 / 64 hours
Fine-tuning	Gradient clipping	1.0
Fine-tuning	Validation / patience	Every 50 steps / 5 checks

B Artifact Traceability

File	Use in this report
<code>energy_forecasting_foundation_models.ipynb</code>	Data loading, model definitions, training, and evaluation
<code>outputs/metrics/leaderboard.csv</code>	Overall ranking and headline metrics
<code>outputs/metrics/*_by_window.csv</code>	Window-level MASE, WQL, and coverage
<code>outputs/metrics/*_per_series.csv</code>	Paired win rates and sign-test counts
<code>outputs/train_logs/</code>	Early-stopping steps and validation curves
<code>outputs/run_meta.json</code>	Hardware, software versions, and observed stop steps
<code>outputs/models/chronos_bolt_base_ft/config.json</code>	Saved Chronos-Bolt architecture

References

- [1] A. F. Ansari, L. Stella, C. Turkmen, et al., “Chronos: Learning the Language of Time Series,” *Transactions on Machine Learning Research*, 2024. <https://openreview.net/forum?id=gerNCVqqtR>
- [2] R. Godahewa, C. Bergmeir, G. I. Webb, R. J. Hyndman, and P. Montero-Manso, “Monash Time Series Forecasting Archive,” *NeurIPS Datasets and Benchmarks*, 2021. <https://arxiv.org/abs/2105.06643>
- [3] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A Time Series is Worth 64 Words: Long-term Forecasting with Transformers,” *International Conference on Learning Representations*, 2023. <https://arxiv.org/abs/2211.14730>
- [4] G. Ke, Q. Meng, T. Finley, et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *Advances in Neural Information Processing Systems*, vol. 30, 2017. https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html
- [5] R. J. Hyndman and A. B. Koehler, “Another Look at Measures of Forecast Accuracy,” *International Journal of Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.